obsidian-mcp-router — référence rapide

v0.97.1 · serveur MCP Obsidian multi-vault + plugin Claude Code qui embarque **et lance** le serveur (54 slash commands · 56 outils MCP · 49 skills)
github.com/tboome33/obsidian-mcp-router

À quoi ça sert, en une minute

Le problème. Vous prenez vos notes dans **Obsidian** (un éditeur de notes en fichiers Markdown, rangés dans des dossiers appelés *vaults*). Vous travaillez avec **Claude Code** (l'assistant de programmation d'Anthropic, en ligne de commande ou dans l'application de bureau). Vous voudriez que Claude lise vos notes, en écrive, cherche dedans — dans *tous* vos vaults, pas un seul — sans lui donner un accès brut à votre disque et sans configurer un serveur par vault.

La réponse. `obsidian-mcp-router` est un **serveur MCP** — MCP (*Model Context Protocol*) est le standard par lequel un assistant comme Claude appelle des « outils » extérieurs. Ce serveur expose 56 outils (lire une note, écrire, chercher, convertir un PDF, construire un graphe des liens…) et **route** chaque appel vers le bon vault : d'où le nom. Il parle à Obsidian par son API HTTP (le plugin communautaire *Local REST API*), jamais en lisant les fichiers directement. Il est livré sous forme de **plugin Claude Code** qui l'installe et le lance : une installation, tout est là.

Ce que vous en faites concrètement. Vous dites « trouve mes notes sur X », « sauvegarde cette conversation dans le wiki », « convertis ce PDF en note », « par où je commence dans ce vault ? » — et Claude enchaîne les bons outils. Le dépôt fournit aussi une méthode de **wiki personnel tenu par l'assistant** (à la manière décrite par Andrej Karpathy : un catalogue, un journal, un cache « chaud » des faits récents, des pages typées) et les commandes pour l'entretenir.

Le vocabulaire, pour lire la suite

`Vault`	Un dossier de notes Markdown ouvert dans Obsidian. Le routeur en connaît plusieurs, **locaux** (sur ce PC) ou **distants** (un Obsidian qui tourne ailleurs, joignable par le réseau). Chaque outil prend un paramètre `vault` ; sans lui, c'est le vault *par défaut* du workspace courant.
`Serveur MCP`	Le programme Node.js de ce dépôt (`bin/obsidian-mcp-router.mjs`). Claude Code le lance et lui parle sur son entrée/sortie standard (« stdio »). C'est lui qui porte les 56 outils.
`Outil MCP`	Une fonction que Claude peut appeler avec des arguments JSON (`get_file` , `write_file` ,

	`search_smart`…). La liste complète est en dernière page.
`Plugin Claude Code (obsidian-router)`	Le paquet installé dans Claude Code depuis le marketplace du dépôt. Il contient le serveur MCP *et* ce qui apprend à Claude à s'en servir : 54 **slash commands**, 49 **skills**, 2 sous-agents, 11 **hooks**.
`Slash command`	Une commande que vous tapez dans Claude Code (`/obsidian-router:meta-status`), ou que Claude reconnaît dans une phrase en français ou en anglais (« diagnostique le router »). Chacune est une procédure écrite pour Claude, qui appelle un ou plusieurs outils.
`Skill`	La page de savoir-faire derrière une commande (`skills/<nom>/SKILL.md`) : la procédure détaillée, les pièges connus, les refus. Claude la charge quand la tâche s'y prête.
`Hook`	Un petit script que Claude Code exécute à un moment précis (au démarrage d'une session, à chaque message…). Le plugin en active 3 d'office : charger le cache chaud du vault, rappeler les décisions déjà prises, dire à quel vault ce workspace est rattaché.
`Local REST API`	Le plugin Obsidian (communautaire) qui ouvre une API HTTP sur chaque vault, avec un port et une clé. **Obligatoire** dans chaque vault : tout ce que le routeur lit ou écrit passe par lui.
`Bridge (mcp-router-bridge)`	Un second plugin Obsidian, écrit pour ce projet, qui ajoute à cette API ce qu'elle n'a pas : la recherche sémantique, l'exécution des templates, l'ouverture d'une note d'un clic, les écritures atomiques. **Optionnel** : sans lui, lire, écrire et chercher du texte marchent toujours.
`Workspace`	Le dossier dans lequel vous avez ouvert Claude Code (souvent un dépôt de code). Le routeur **lie** chaque workspace à un vault principal, éventuellement à des secondaires : c'est ce qui décide où atterrit une écriture faite sans nommer de vault.
`config.json`	`~/.claude/obsidian-mcp-router/config.json` : la liste de vos vaults (chemin, port, clé), les vaults distants, les liaisons de workspaces. C'est *votre* fichier, sur votre machine — jamais dans un dépôt.
`Wiki, wiki-meta/`	La structure de notes que les commandes `wiki-*` entretiennent dans un vault : les pages sous `wiki/`, et sous `wiki-meta/` quatre fichiers d'échafaudage —

	`catalog.md` (l'index), `journal.md` (l'historique), `hot.md` (le cache des faits récents, réécrit, jamais empilé), `overview.md` (la vue d'ensemble).
`Conventions`	Des sections du fichier `CLAUDE.md` d'un vault qui disent à Claude comment écrire *dans ce vault* (langues, provenance des sources, structure des titres…). Installables à l'unité depuis une bibliothèque de 14 snippets.
`Smart Connections, Templater`	Deux plugins Obsidian tiers. Le premier calcule des *embeddings* (des vecteurs qui rapprochent les notes par sens) : le routeur s'en sert pour la recherche sémantique et la détection de pages jumelles. Le second exécute des modèles de notes.
`OKF`	*Open Knowledge Format* (v0.1, Google) : un format d'échange de bases de notes. Le routeur exporte un wiki en bundle OKF, valide un bundle, et génère la navigation OKF (index par dossier) dans le vault.

Comment les briques s'emboîtent — la chaîne de dépendances

```
Obsidian  ←  Local REST API (plugin communautaire)  ←  BRIDGE (mcp-router-bridge)
    ↑ HTTP par vault (port + apiKey du plugin Local REST API)
SERVEUR MCP (obsidian-mcp-router) — process Node sur le PC
    ↑ MCP sur stdio, spawné par Claude Code
PLUGIN CLAUDE CODE (obsidian-router) — commands + skills + agents + hooks
```

Lire le schéma de bas en haut. Vous parlez à Claude Code. Le plugin lui a appris les commandes et lance le serveur MCP. Le serveur reçoit un appel d'outil, choisit le vault, et envoie une requête HTTP au plugin Local REST API de cet Obsidian — ou au bridge, pour les capacités que seul un Obsidian *en marche* peut fournir.

Pourquoi HTTP plutôt que lire les fichiers sur le disque ? Parce que le routeur parle à un Obsidian *vivant*, pas à du Markdown inerte. Ça lui donne ce que seule l'application qui tourne possède : les embeddings de Smart Connections (recherche par le sens, `search_smart`), le rendu réel d'un template Templater (`execute_template`), l'ouverture d'une note dans la fenêtre (click-to-open). Une écriture passe par le pipeline de modification d'Obsidian lui-même, au lieu de risquer d'écraser un fichier qu'un humain édite au même moment. Et la même requête HTTP atteint un vault sur un NAS ou derrière un tunnel WireGuard exactement comme un vault local. Le prix : Obsidian doit tourner, et chaque vault a un port et une clé API à suivre — c'est ce que `config.json` et `setup-vault` gèrent pour vous.

Bridge — tourne *dans* Obsidian ; il greffe ses routes sur le serveur HTTP de Local REST API : `/search/smart` (recherche sémantique), `/templates/execute` (Templater), `/open/*` et `/ping` (click-to-open et auto-test « quel vault répond sur ce port »), `PUT /vault-cas/*` (≥

0.7.0 — écritures conditionnelles *atomiques* : « n'écris que si le fichier n'a pas changé depuis ma lecture »), `GET /smart-env/sources` (≥ 0.9.0 — sert le magasin de vecteurs de Smart Connections, un dossier caché que l'API REST refuse de servir elle-même, ce qui rend l'analyse de pages jumelles possible sur un vault distant). **Optionnel** : sans lui, le CRUD de base marche ; les écritures `ifMatch` retombent sur un repli vérifié mais non atomique. Il se met à jour via BRAT depuis les releases GitHub.

Serveur MCP — process Node ≥ 20.19.0 sur le PC, lancé par Claude Code (via le plugin — le cas normal — ou via une entrée manuelle dans `~/.claude.json`, pour le développement). Requiert le plugin **Local REST API** dans **chaque** vault ; son registre de vaults vit dans `~/.claude/obsidian-mcp-router/config.json`.

Plugin Claude Code — ses commandes et ses skills orchestrent les outils du serveur ; deux de ses hooks lisent le vault directement sur disque, parce qu'ils tournent avant tout appel d'outil (`hot-cache-load`, `decisions-recall`). Il **embarque et lance le serveur** : une installation = tout, une mise à jour = tout.

Ce que le routeur sait faire — les grandes familles, et pourquoi elles existent

- **Un seul serveur pour tous vos vaults.** Chaque outil prend un paramètre `vault` ; `vault: "*"` lance une recherche sur tous les vaults à la fois (« fan-out »). Fini un serveur MCP par vault.
- **Vaults locaux et distants traités pareil.** Un vault distant, c'est une URL et une clé API dans `config.json` — Obsidian peut tourner dans un conteneur sur un serveur, sur une autre machine du réseau. Depuis la v0.97.0, un vault distant dont les fichiers sont *aussi* visibles d'ici (un partage monté) peut le déclarer (`localPath`), ce qui permet d'installer ses plugins et de lire son cache chaud depuis cette machine.
- **Liaison workspace → vault.** Quel vault un dépôt de code ouvre-t-il ? Ce n'est plus son propre `.env` qui le décide (un dépôt cloné transporterait la décision de quelqu'un d'autre) : c'est une liaison confirmée une fois, sur *votre* machine, dans `config.json` (`confirm_workspace_binding`, ou `setup-vault --attach`). Le `.env` d'un projet ne peut que **proposer** ; le briefing de début de session dit à quoi ce workspace est rattaché et comment en changer.
- **Mode lock.** Restreint une session à un seul vault (sécurité, concentration) : tout appel vers un autre vault est refusé.
- **Auto-enrichissement du wiki.** À trois moments naturels d'une conversation (une réponse substantielle, une décision prise, un changement de sujet), Claude propose d'archiver dans le wiki. Quatre modes : `ClaudeAsk` (il demande, le défaut) · `Hybrid` · `FullAuto` (il sauvegarde sans demander) · `off`.
- **Boîte à outils de conversion.** PDF, DOCX, XLSX, PPTX, image (OCR), audio (transcription), vidéo YouTube (sous-titres), page web, dépôt git → Markdown, prêt à être écrit dans une note. Le moteur principal, **MarkItDown**, est un extra en Python à installer une fois (`npm run install-markitdown`) ; sans lui, les outils restent visibles et renvoient l'instruction d'installation. **Docling**, second extra (`npm run install-docling`), fait le PDF haute fidélité (tableaux, mise en page) et `pdf_to_images` rend des pages en images pour

que le modèle les *voie*. Un **filtre de pertinence** (BM25, la formule classique des moteurs de recherche) retire les blocs hors sujet avant synthèse.

- **Graphe de connaissances.** Le routeur construit un graphe JSON typé des liens entre pages, en tire un parcours de lecture (« par où commencer »), les voisines d'une page, le plus court chemin entre deux pages ; il exporte le wiki en `llms.txt` ou en bundle OKF.
- **Analyse du wiki.** `find_boundary_pages` repère les pages « frontière » — très liées mais presque vides, celles qui méritent d'être écrites ; `find_twin_pages` repère les paires de pages quasi identiques par similarité de vecteurs, avec un seuil calculé sur *votre* vault — et marche aussi sur un vault distant (bridge ≥ 0.9.0).
- **Click-to-open.** Chaque écriture renvoie un lien ; un clic ouvre la note dans Obsidian (route `GET /open/<path>` du bridge). `open_in_obsidian` fait pareil sans navigateur (idéal dans Claude Desktop). Pour un vault dont le disque n'est pas visible, le port du lien se déclare dans la config (`insecurePort`), et `GET /ping?v=<vault>` prouve d'un clic *quel* vault répond sur un port.
- **Assistant de création de vault.** `plan_vault` (un plan lisible, avec les défauts et les avertissements) → ajustements → `provision_vault` (plugins, port, clé, wiki, registre) : dans le cas simple, une seule interaction.
- **Mode multi-tenant** (facultatif) : quand une instance du routeur sert plusieurs personnes (derrière un proxy, sur MCPHub), des variables d'environnement la cadrent — liste blanche de vaults, lecture seule, piste d'audit par utilisateur.
- **11 hooks.** 3 actifs d'office pour tout installeur du plugin (`hot-cache-load`, `decisions-recall`, `workspace-briefing`, déclarés dans `hooks/hooks.json`) ; les 8 autres sont à activer (câblés en fin de `setup-vault`, `--no-hooks` pour s'en passer, `node scripts/setup-vault.mjs --install-hooks` pour les ajouter plus tard) : journal automatique de session, garde de taille du cache chaud, autocommit du wiki, vérificateur de liens, détecteur de dérive de la doc, vérification de mise à jour…

Installer, puis attacher un vault — le parcours normal

1. **Installer le plugin `obsidian-router` dans Claude Code.** C'est tout : le serveur MCP est livré dedans et lancé automatiquement. (Le clone du dépôt + `npm link`, via `/obsidian-router:meta-setup`, est le parcours des développeurs du routeur.)
2. **Avoir Node.js ≥ 20.19.0.** Python 3.10+ n'est requis que pour les extras de conversion (`npm run install-markitdown`, `npm run install-docling`).
3. **Savoir si ces extras sont là *avant* qu'un appel échoue.** Chaque réponse de `list_vaults` porte `conversionToolbox` (`available`, `via`, `verified`, `hint`) et `meta-status` l'affiche en une ligne. Huit outils s'arrêtent sans MarkItDown ; `youtube_to_markdown` se rabat sur les sous-titres via yt-dlp (s'il est installé) ; `git_repo_to_markdown` (repomix) et `pdf_to_markdown_docling` ne sont pas concernés. Pour ne plus se le faire proposer : `OBSIDIAN_ROUTER_SKIP_MARKITDOWN=1`.
4. **Le chemin le plus simple : `/obsidian-router:meta-attach-vault`.** Un assistant interactif qui attache un vault au workspace courant (le cas courant), crée un vault autonome, ou enregistre un vault distant. Il installe les plugins Obsidian nécessaires, met

en place le wiki, écrit le `.env`, propose les conventions à installer, et termine par un **état final vérifié** : plugins, wiki, conventions — et la liste ordonnée de ce qui manque, s'il manque quelque chose.

5. **Alternative en ligne de commande :**

   ```
   npm run setup-vault -- "C:\VAULTS\MonVault"
   ```

   installe et configure les plugins **Local REST API** + bridge, écrit le `.env`, alloue un port unique, ajoute le vault au registre, câble les hooks (`--no-hooks` pour s'en passer, `--install-hooks` pour le faire plus tard). `--attach <nom>` lie le workspace courant à un vault déjà connu ; pour un vault distant dont les fichiers sont visibles d'ici, `--attach <nom> --local-path <dossier>` vérifie que le dossier est bien ce vault avant de l'enregistrer.
6. **Le bridge** (`obsidian-mcp-router-bridge`) est requis pour la recherche sémantique (avec Smart Connections), l'exécution de templates (avec Templater) et les liens click-to-open. Sans lui : lire, écrire, chercher du texte fonctionnent ; ces trois capacités non. `--install-plugins <vault> --dry-run` montre ce qui manque, puis l'installe depuis les releases GitHub officielles avec le sceau que l'aperçu a imprimé ; `--plugin-health <vault>` dit ce qu'Obsidian a réellement chargé.
7. **Vérifier :** `/obsidian-router:meta-status`, ou dire simplement *« diagnostique le router »*.

Configuration — le `.env` du workspace (les SEULES clés que le routeur prend dans le fichier d'un dépôt ; tout le reste y est ignoré et nommé sur sa sortie d'erreur)

Pourquoi cette règle. Un `.env` voyage avec un dépôt cloné : ce qu'il contient a été écrit par quelqu'un d'autre, sur une autre machine. Le routeur n'y prend donc que des *préférences de lecture* et des *propositions* ; tout ce qui accorde un droit (écrire sans demander, verrouiller, cadrer une instance) vient de l'hôte MCP ou de votre `config.json`.

`VAULT_PATH`	**À quoi ça sert :** dire quel vault ce workspace utilise, de façon portable. Chemin du vault courant ; s'il correspond à un vault enregistré, ce vault devient le défaut. Écrit par `setup-vault.mjs`.
`OBSIDIAN_ROUTER_DEFAULT_VAULT`	**À quoi ça sert :** choisir le vault par défaut d'un process. **Depuis l'hôte uniquement** (déclaration du serveur MCP, lanceur, shell), où il gagne sur `VAULT_PATH` et `config.defaultVault`. La même variable dans le `.env` d'un projet est une **proposition** : rapportée par `list_vaults` dans `bindingHint`, jamais appliquée. Ce qui décide, c'est la liaison confirmée du workspace dans votre `config.json` (`workspaceBindings`) — `confirm_workspace_binding` ou `setup-vault --attach` l'enregistre.

`OBSIDIAN_ROUTER_LOCKED`	**À quoi ça sert :** travailler dans un seul vault, sans risque d'écrire ailleurs. Tout appel vers un autre vault échoue. Posé via `/obsidian-router:lock <vault> --persist`. Même règle d'origine : autorité depuis l'hôte, proposition depuis un fichier de projet. `--persist` enregistre `locked: true` sur la liaison (relu au redémarrage) ; la ligne `.env` qu'il écrit aussi est un indice portable pour une autre machine.
`OBSIDIAN_ROUTER_AUTO_ENRICH`	**À quoi ça sert :** régler à quel point Claude archive de lui-même dans le wiki. `ClaudeAsk` (défaut) | `Hybrid` | `FullAuto` | `off`. Posé via `/obsidian-router:auto-mode <Mode> --persist`. **Une valeur est refusée depuis un fichier de workspace :** tout ce qui se lit `FullAuto`, parce que c'est une autorisation permanente d'écrire sans redemander, et qu'un `.env` cloné n'a pas à l'accorder. `FullAuto` vient de la déclaration du serveur dans l'hôte, ou d'un appel à `set_auto_enrich_mode` dans la session ; un refus est nommé sur la sortie d'erreur et porté par `list_vaults` (`autoEnrichModeRefused`).
`MD_ALLOWED_PATHS · MD_SHARE_DIR`	**À quoi ça sert :** limiter ce que les outils de conversion peuvent lire et où ils peuvent écrire sur le disque local (un bac à sable). Absent = pas de bac à sable. Les deux noms sont UN réglage ; un fichier de workspace ne peut que le *resserrer*.
`OBSIDIAN_ROUTER_NO_*`	**À quoi ça sert :** couper une commodité (jamais un garde). Les 14 opt-outs énumérés dans `src/helpers/workspace-dotenv.mjs`, acceptés par nom : NO_WATCH (rechargement à chaud de la config), NO_HOT_CACHE_LOAD, NO_DECISIONS_RECALL, NO_WIKI_QUERY_FIRST, NO_WIKI_AUTOCOMMIT, NO_SESSION_JOURNAL, NO_HOT_CACHE_GUARD, NO_LINT_VAULT_LINKS, NO_DOC_STARTUP_CHECK, NO_DOC_PROPAGATION_CHECK, NO_UPDATE_CHECK, NO_AUTO_INSTALL_HOOKS, NO_AUTO_CONFORMANCE, NO_OKF_PROJECTIONS. Valeurs : `1` / `true` / `yes` / `on`.

Configuration — variables d'hôte (jamais prises dans un `.env` de workspace : à poser dans la déclaration du serveur côté hôte MCP, dans le lanceur d'une instance servie, ou dans le shell)

`OBSIDIAN_ROUTER_ALLOWED_VAULTS`	**À quoi ça sert :** ne montrer à une instance qu'une partie des vaults (déploiement partagé). Liste blanche,

	séparée par des virgules ; les autres sont ignorés.
`OBSIDIAN_ROUTER_READONLY`	**À quoi ça sert :** une instance qui ne peut rien modifier. Valeur vraie (`true` / `1` / `yes` / `on`) : les **18 outils d'écriture** (write_file, append_to_file, patch_file, set_frontmatter, merge_frontmatter, move_file, delete_file, execute_template, download_page_assets, pptx_extract_assets, build_wiki_graph, provision_vault, register_remote_vault, write_bundle, record_source, install_conventions, refresh_okf_projections, build_search_index) sont retirés de la liste des outils *et* refusés à l'appel.
`OBSIDIAN_ROUTER_USER_ID`	**À quoi ça sert :** savoir qui a écrit quoi quand plusieurs personnes passent par la même instance. Chaque écriture réussie ajoute une ligne attribuée au `wiki-meta/journal.md` du vault touché. Masque aussi les outils réservés au poste local (`plan_vault` , `provision_vault`).
`VAULT_<NOM>`	**À quoi ça sert :** déclarer un vault sans fichier de config — pratique sur un serveur administré par un tableau de bord (MCPHub). Un vault entier en JSON : `name` , `baseUrl` , `apiKey` , et pour un vault distant `obsidianName` (son nom exact dans Obsidian, pour les liens d'ouverture).
`OBSIDIAN_ROUTER_CONFIG · MARKITDOWN_PATH · DOCLING_PATH · REPOMIX_PATH · YTDLP_PATH · HTTPS_PROXY · HF_HOME…`	**À quoi ça sert :** dire où sont la config et les outils externes. Lus dans l'environnement de l'hôte uniquement (README § environment). Un outil lancé reçoit une liste nommée de variables, jamais l'environnement entier du routeur.

Fichiers importants — où vivent les choses

`~/.claude/obsidian-mcp-router/config.json`	La config principale du routeur : vaults et registre de ports, vault par défaut, vaults désactivés, vaults distants, liaisons de workspaces. Rechargée à chaud quand elle change.
`~/.claude.json`	Parcours développeur uniquement. Le plugin déclare le serveur (clé `router`) dans son `.mcp.json` racine via `${CLAUDE_PLUGIN_ROOT}` . Une entrée manuelle `mcpServers.obsidian-router` en plus fait tourner **deux process** aux outils dupliqués (certains postes de dev le font exprès) ; `claude --plugin-dir <checkout>` remplace le plugin installé le temps d'une session.

`~/.claude/settings.json`	Les **8 hooks à activer** (sur 11) se déclarent ici ; les 3 actifs d'office viennent de `hooks/hooks.json` dans le plugin. Le plugin s'active **par workspace** via `<workspace>/.claude/settings.json` — 54 commandes + 49 skills ≈ 10 k tokens de contexte, à ne charger que là où vous utilisez Obsidian.
`<workspace>/.env`	Lu au démarrage du routeur — mais seules les clés de workspace ci-dessus sont prises, l'environnement parent gagne toujours, et toute autre clé est ignorée et nommée une fois sur la sortie d'erreur.
`<vault>/CLAUDE.md`	Les consignes du vault, chargées en contexte par Claude Code quand la session est ouverte *dans* le vault : le mode d'auto-enrichissement et les conventions installées. Depuis un workspace de code, elles ne sont pas chargées : la première écriture du routeur dans le vault les renvoie (`vaultConventions`).
`<vault>/wiki-meta/{catalog,journal,hot,overview}.md`	L'échafaudage du wiki : catalogue, journal (ajouts seulement), cache chaud, vue d'ensemble — sous `wiki-meta/` ; les pages elles-mêmes sont sous `wiki/`. Nommés `catalog` / `journal` plutôt qu'`index` / `log` parce qu'OKF réserve ces deux noms ; les anciens restent lus. Créés par `/obsidian-router:wiki`.
`<vault>/wiki-meta/Sessions/`	Le détail de chaque session (écrit par le hook `session-auto-journal`) ; `journal.md` n'en garde qu'une ligne.
`obsidian-mcp-router/docs/`	`auto-enrichment.md` (les 4 modes et les 4 canaux de rangement), `features/` (13 fiches en prose), `architecture.md`, `remote-vaults.md` (vaults distants), cette page (FR + EN).

54 slash commands /obsidian-router:<nom> · reconnues aussi en langage naturel (FR + EN)

Une slash command est une procédure écrite pour Claude. Vous n'avez pas besoin de la taper : la troisième colonne donne une phrase qui la déclenche. Elles se répartissent en cinq groupes selon *ce que vous voulez faire*.

🔧 17 wrappers MCP — une commande par outil de base du vault

Quand : vous voulez une opération précise sur une note — lire, écrire, chercher, déplacer, convertir. Chaque wrapper appelle un outil et vous montre le résultat. Catégories : `discover` · `read` · `write` · `manage` · `template` · `convert`

COMMANDE	EFFET	PHRASE DÉCLENCHEUSE (FR/EN)
`discover-list-vaults`	Liste les vaults (en ligne ou non, latence, état du lock)	« liste mes vaults » / "list my vaults"
`discover-list-files`	Liste les fichiers d'un dossier de vault	« liste les fichiers de X » / "list files in Sessions"
`read-get`	Lit un fichier (Markdown + frontmatter)	« montre-moi X » / "show me X"
`read-search`	Recherche de texte simple (sous-chaîne)	« trouve <texte> » / "grep for <text>"
`read-search-smart`	Recherche par le sens (Smart Connections)	« trouve mes notes sur X » / "find notes about X"
`read-frontmatter`	Lit le frontmatter (tout, ou une clé)	« quel est le statut de X » / "metadata of X"
`write-create-or-replace`	Crée ou remplace un fichier	« crée une note X » / "save this as X.md"
`write-append`	Ajoute à la fin d'un fichier	« ajoute à X » / "append to my journal"
`write-patch`	Modification chirurgicale (un titre, un bloc, le frontmatter)	« édite la section X dans Y » / "edit X section in Y"
`write-frontmatter-set`	Pose une clé de frontmatter	« passe le statut de X à closed » / "tag this with X"
`write-frontmatter-merge`	Pose plusieurs clés d'un coup	« sur X mets status=closed outcome=tp1 »

`write-bundle`	Plusieurs fichiers d'un bloc — tout ou rien, avec retour arrière journalisé	« écris ces 4 pages d'un bloc » / "write these pages atomically"
`manage-move`	Déplace ou renomme un fichier	« renomme X en Y » / "move X to Y"
`manage-delete`	Supprime un fichier (confirmation en deux temps)	« supprime X » puis « oui confirm=true »
`template-execute`	Exécute un template Templater	« exécute le template daily » / "run X template"
`pdf-to-markdown`	PDF local → Markdown (MarkItDown, texte brut, rapide)	« convertis ce PDF en markdown » / "convert this PDF"
`pdf-to-markdown-docling`	PDF → Markdown haute fidélité (Docling : tableaux, mise en page ; extra à installer)	« conversion haute fidélité » / "convert with docling"

🔒 4 commandes d'état du routeur (verrou, auto-enrichissement, base de ports)

Quand : vous voulez changer comment le routeur se comporte pendant cette session — dans quel vault il travaille, à quel point il archive seul, quels ports il attribuera aux prochains vaults.

COMMANDE	EFFET	PHRASE DÉCLENCHEUSE (FR/EN)
`lock`	Restreint la session à un vault (le temps de la session, ou `--persist`)	« verrouille sur X » / "I only want to work on X"
`unlock`	Lève le verrou, rétablit le routage multi-vault	« déverrouille les vaults » / "unlock vaults"
`auto-mode`	Règle le mode d'auto-enrichissement (ClaudeAsk / Hybrid / FullAuto / off)	« passe en mode Hybrid » / "switch to Hybrid mode"
`force-new-port-start`	Tire une nouvelle base d'attribution de ports pour les **futurs** vaults uniquement — plan scellé en deux temps, aucun port existant modifié	« tire une nouvelle base de ports » / "draw a new port base"

🔗 2 commandes de liaison de workspace (dans quel vault ce projet écrit)

Quand : vous ouvrez Claude Code dans un dossier de projet et voulez que « le vault » désigne le bon, sans le nommer à chaque appel — et décider ce que ce projet a le droit d'écrire dans d'autres vaults.

COMMANDE	EFFET	PHRASE DÉCLENCHEUSE (FR/EN)
`bind-workspace`	Assistant — rattache ce workspace à un vault PRINCIPAL, puis éventuellement à des secondaires avec un palier d'écriture chacun ; nomme aussi une proposition du `.env` restée sans réponse, et enregistre un « non » comme un refus	« à quel vault ce projet est-il rattaché » / "bind this workspace to a vault"
`configure-secondary-vaults`	Le même assistant, entré à l'étape des SECONDAIRES : déclare les secondaires et choisit chaque palier — lecture seule stricte / écriture sur demande / lecture-écriture	« ajoute un vault secondaire » / "make X read-only for this project"

🩺 7 helpers conversationnels (installation + diagnostic)

Quand : quelque chose est à installer, à vérifier ou à réparer — un vault à attacher, un plugin manquant, un vault injoignable, des conventions à propager.

COMMANDE	EFFET	PHRASE DÉCLENCHEUSE (FR/EN)
`meta-setup`	Installation manuelle du routeur (clone + npm link) — parcours développeur ; une installation normale reçoit le serveur par le plugin	« installe le router » / "setup obsidian-mcp-router"
`meta-attach-vault`	Assistant — attache un vault à un workspace (le cas courant), crée un vault autonome, ou enregistre un vault distant ; termine par l'état final vérifié	« attache un vault à ce workspace » / "connect my remote vault"
`meta-status`	Bilan de santé de tous les vaults, avec des pistes de correction ; dit aussi quelle version du routeur répond et	« diagnostique le router » / "are my vaults reachable"

	si les hooks de la session ont tourné	
`meta-sync-template`	Propage plugins, snippets et docs du vault de référence vers d'autres vaults (sélection interactive)	« synchronise le template » / "sync the template to all vaults"
`sync-from-github`	Met à jour un vault ou toute la flotte depuis le squelette publié sur GitHub (plugins, thèmes, snippets, docs) — sans dépôt de développement local	« synchronise depuis GitHub » / "sync this vault from github"
`meta-audit-bridge-readiness`	Vérifie que le click-to-open est prêt (bridge, API REST, HTTP, sonde `/open` en direct)	« audite le bridge » / "is click-to-open ready"
`conventions`	Installe / retire / liste / audite / propage les conventions du `CLAUDE.md` d'un vault ; migre l'ancienne convention `bilingual` vers `languages`	« installe la convention X » / "install source-type on X"

📚 24 commandes de gestion de connaissances (le wiki tenu par l'assistant)

Quand : vous construisez et entretenez une base de notes avec Claude — y verser des sources, l'interroger, la vérifier, la compacter, la visualiser, l'exporter.

COMMANDE	EFFET	PHRASE DÉCLENCHEUSE (FR/EN)
`wiki`	Met en place le wiki dans un vault (catalogue, journal, cache chaud, vue d'ensemble) et propose les conventions	« scaffold un wiki » / "set up a wiki"
`wiki-ingest`	Verse une source (URL, fichier, texte) dans le wiki → pages d'entités et de concepts, liées entre elles	« ingère cette URL » / "absorb this article"
`wiki-query`	Répond à partir du wiki (trois niveaux de recherche, sans le web)	« que dit mon wiki sur X » / "based on my notes …"
`wiki-lint`	Bilan de santé : pages orphelines, liens morts, catalogue en retard	« lint le wiki » / "audit my wiki"
`wiki-fold`	Compacte le journal en résumés périodiques sous `wiki/folds/` (rejouable sans doublon)	« compacte le journal » / "fold the log"
`hot-compact`	Ramène un `hot.md` trop gros à son contrat de cache (sauvegarde vérifiée d'abord)	« compacte le hot » / "compact the hot cache"
`save`	Archive la conversation en note typée (session, décision, ADR…)	« sauvegarde ça » / "save this"
`decision-consolidate`	Réduit une page de décision tranchée à l'essentiel et déplace la délibération complète dans une note d'archive vérifiée	« consolide cette décision » / "consolidate this decision"
`autoresearch`	Boucle autonome : recherche web → synthèse → archivage,	« fais une recherche web sur X » / "research X online"

	jusqu'à épuisement des questions	
`canvas`	Crée ou modifie des fichiers `.canvas` Obsidian (tableaux visuels)	« crée un canvas pour X » / "create a canvas for X"
`defuddle`	Retire le bruit d'une page web (menus, publicités) avant de l'ingérer	« nettoie cette page » / "defuddle <url>"
`obsidian-bases`	Crée ou modifie des fichiers `.base` Obsidian (vues de type base de données)	« crée une base pour X » / "create a base for X"
`wiki-graph`	Construit le graphe des liens (JSON typé, lisible par le visualiseur natif d'Obsidian)	« construis le graphe » / "build the wiki graph"
`wiki-tour`	Un parcours de lecture ordonné, déduit de la topologie des liens	« par où je commence » / "give me a tour of this vault"
`wiki-neighbors`	Les voisines d'une page — liens sortants, liens entrants, ou les deux	« voisins de X » / "what links to X"
`wiki-path`	La plus courte chaîne de liens entre deux pages	« quel rapport entre X et Y » / "how is X connected to Y"
`wiki-export`	Exporte le vault en `llms.txt` / `llms-full.txt` ou en bundle OKF	« exporte le wiki » / "export the wiki as llms.txt"
`okf-export`	Exporte une partie du wiki en bundle OKF v0.1 partageable (conformité vérifiée)	« publie en bundle OKF » / "export this folder as an OKF bundle"
`okf-projections`	Régénère la navigation OKF dans `wiki/` (index racine et par dossier, journal du plus récent au plus ancien) ; rafraîchie automatiquement après les écritures ; `--check` = rapport de dérive	« regénère les index du wiki » / "refresh the OKF projections"
`okf-check`	Valide n'importe quel bundle OKF contre les règles de conformité v0.1	« ce bundle est-il conforme ? » / "validate this OKF bundle"

`wiki-refresh-digests`	Régénère les résumés annexes de chaque page (concepts, affirmations, mots-clés)	« rafraîchis les digests » / "refresh the digests"
`build-search-index`	Construit ou rafraîchit l'index de recherche BM25 local (sans plugin ; rejouable)	« construis l'index de recherche » / "build the search index"
`wiki-boundary`	Classe les pages « frontière » — très liées mais presque vides, celles qui méritent d'être écrites	« qu'est-ce que je devrais écrire ensuite » / "frontier pages"
`who-is-speaking`	Dans un vault familial partagé, identifie qui parle et range ses notes dans son dossier	« c'est Karine » / "who is speaking"

Les 56 outils MCP — ce que Claude appelle réellement, et à quoi chacun sert

Vous n'appelez pas ces outils vous-même : les commandes et les skills le font. Cette table sert à comprendre ce qui se passe, et à lire les réponses. Chaque outil accepte `vault` ; les écritures acceptent `ifMatch` (voir « Écritures »).

`list_vaults` · `list_files`	**Découverte — savoir ce qui existe.** `list_vaults` est l'appel à faire en premier : le catalogue des vaults, en ligne ou non, avec la latence, la version du routeur qui répond (`routerBuild`), si les hooks de la session ont tourné (`sessionHooks`), l'état des extras de conversion (`conversionToolbox`) et la liaison du workspace. `list_files` liste un dossier.
`get_file` · `search` · `search_smart` · `get_frontmatter`	**Lectures.** Le fichier complet (renvoie `contentSha256`, l'empreinte à rejouer en `ifMatch` pour écrire sans écraser) ; recherche de texte ; recherche par le sens (embeddings Smart Connections, scores cosinus) ; frontmatter typé. `vault: "*"` = sur tous les vaults. `search_smart` dit deux choses sur sa propre réponse : `freshness` signale un résultat dont la page a changé depuis son indexation (`changed` / `touched`) ou a disparu (`page-missing`) — ouvrez la page avant de la citer ; `folderExclusion` dit ce qui a été laissé de côté (par défaut `wiki-meta/Sessions`, les journaux chronologiques ; `excludeFolders: []` les remet).
`write_file` · `append_to_file` · `patch_file` · `delete_file` · `set_frontmatter` · `merge_frontmatter` · `move_file`	**Écritures et gestion de fichiers.** Créer ou remplacer ; ajouter à la fin ; modifier un titre, un bloc ou le frontmatter ; supprimer (gardé par `confirm: true`) ; une ou plusieurs clés de frontmatter ; déplacer ou renommer. **`ifMatch`** : rejouez le `contentSha256` lu par `get_file` et l'écriture est refusée (409) si le fichier a changé entre-temps — c'est ce qui protège une note que quelqu'un d'autre édite ; atomique avec bridge ≥ 0.7.0, repli vérifié sinon. Un vault partagé entre plusieurs workspaces l'exige (`writesRequireIfMatch`).
`execute_template`	**Templater.** Rend un template et écrit le résultat si demandé ; les valeurs sont passées via `tp.mcpTools.prompt("clé")`.

`lock_vault` · `unlock_vaults` · `set_auto_enrich_mode` · `confirm_workspace_binding` · `set_secondary_vault_mode`	**État du routeur et liaison du workspace.** Isolation à un seul vault ; mode d'auto-enrichissement ; confirmer, refuser ou effacer la liaison de ce workspace (`{ vault }` , `{ vault, also: [...] }` pour plusieurs, `{ refuse: true }` , `{ clear: true }`) — enregistrée dans *votre* `config.json` , jamais dans le projet. `set_secondary_vault_mode` règle le **palier d'écriture** d'un secondaire : `locked` (refus côté serveur), `soft` (le défaut : refusé tant qu'une confirmation `confirmSecondaryWrite` n'a pas été donnée), `writable` . Sur une instance partagée (`READONLY` / `ALLOWED_VAULTS` / `USER_ID`), refusés : une réponse persistée parlerait pour tout le monde.
`plan_vault` · `provision_vault` · `register_remote_vault`	**Créer ou déclarer un vault.** Un plan en lecture seule (défauts, questions, avertissements) → création en un appel (plugins, port, clé, wiki, registre) — le vault est jugé sur son chemin réel, épinglé pour toute l'opération, et refusé si son arbre contient un lien. `register_remote_vault` enregistre depuis la conversation un vault distant déjà en service (nom, `baseUrl` , `apiKey` , et si ses fichiers sont visibles d'ici `localPath`) — dans `config.json` , jamais dans le `.env` d'un projet ; caché sur une instance partagée.
`pdf/docx/xlsx/pptx/image/audio_to_markdown`	**Conversion locale** (MarkItDown, extra à installer : `npm run install-markitdown` ; les outils restent listés et renvoient l'instruction d'installation s'il manque). Bureautique, PDF, OCR d'image, transcription audio → Markdown, à enchaîner avec `write_file` .
`pdf_to_markdown_docling` · `pdf_to_images`	**PDF haute fidélité** (Docling, extra) : mise en page et tableaux préservés. `pdf_to_images` rend des pages en PNG pour que le modèle les *voie* (nombre de pages et octets plafonnés).
`pptx_extract_assets`	**Les images d'une présentation, sur disque**, sans Python (un PPTX est un ZIP) : chaque image embarquée écrite une fois, avec les diapositives qui l'affichent — ce que `pptx_to_markdown` ne peut pas rendre. Outil d'écriture : un `outdir` dans un vault subit le

	palier de ce vault ; hors de tout vault, seul le dossier temporaire est permis.
`youtube/bing_search/webpage_to_markdown` · `git_repo_to_markdown`	**Conversion distante.** URL → Markdown (avec une garde contre les adresses internes) ; sous-titres YouTube ; un dépôt git entier → un seul document (repomix, compression Tree-sitter en option). `webpage_to_markdown` accepte `relevanceQuery` (ne garder que ce qui répond à une question) et `citations: true` (les liens deviennent des notes de bas de page numérotées).
`filter_relevant_blocks`	**Filtre de pertinence.** Une seconde passe BM25, déterministe, sur un Markdown déjà obtenu : retire les blocs hors sujet, garde frontmatter et titres. Aucun accès réseau, aucun modèle de langage.
`extract_page_metadata` · `propose_linked_sources` · `download_page_assets`	**Aide à l'ingestion web.** Les métadonnées d'une page (JSON-LD, OpenGraph) ; les liens qui valent d'être suivis, classés ; le téléchargement des images dans le vault.
`get_wiki_context_pack` · `build_wiki_graph` · `build_wiki_tour` · `get_page_neighbors` · `wiki_path`	**Contexte et graphe.** Un paquet de contexte structuré pour un agent extérieur ; le graphe typé ; le parcours de lecture ; les voisines ; le plus court chemin. Chaque élément du paquet dit d'où il vient : `index` (le catalogue) et `graph` (un lien) sont de la **navigation, qui fait foi** ; `semantic` est un complément, jamais l'unique appui d'un fait. Un paquet sans ancrage de navigation le dit : `answer-relies-on-semantic-only`.
`find_boundary_pages` · `find_twin_pages`	**Analyse du wiki.** Les pages « frontière » qui méritent d'être écrites ; les paires quasi jumelles (similarité de vecteurs, seuil calculé sur le vault), chacune avec les indices pour trancher. Quand ils ne peuvent pas regarder, les deux répondent `available: false` avec la raison — jamais un faux « 0 trouvé ». `find_twin_pages` marche aussi sur un vault distant (bridge ≥ 0.9.0).
`write_bundle` · `refresh_okf_projections` · `build_search_index`	**Entretien du wiki.** Plusieurs fichiers d'un bloc, tout ou rien, avec retour arrière ; régénération de la navigation OKF (`index.md` par dossier, `log.md` du plus récent au plus

	ancien — fichiers générés, à ne jamais éditer à la main) ; index de recherche BM25 local (tous les vaults, aucun plugin requis).
`record_source` · `audit_sources`	**Registre des sources.** D'où vient ce qui a été ingéré (URL, empreinte, date) ; audit des sources d'un vault (dérives, trous).
`audit_vault_conventions`	**Audit des conventions.** Sous quelles conventions un vault est réellement : le fichier que le routeur retient (ou les deux entre lesquels il ne peut pas choisir), les recommandées absentes, les copies du modèle, les sauvegardes héritées — chaque constat avec une réparation à approuver, jamais appliquée seule.
`install_conventions`	**Installateur de conventions.** Installe des conventions de la bibliothèque, par identifiant, dans le fichier de conventions du vault en UNE écriture protégée (textes lus dans le paquet, jamais recopiés par le modèle), puis relit le fichier et vérifie ; `languages` prend la valeur propre au vault, une convention retirée est signalée et jamais écrite, `dryRun` rend l'état dont le menu de sélection a besoin.
`build_open_link` · `open_in_obsidian` · `get_view_link`	**Navigation.** Un lien click-to-open prêt à coller (chemin vérifié contre le vault ; sur un vault sans disque, le lien est construit quand même, marqué `pathVerified: false`) ; ouverture côté serveur (la fenêtre Obsidian passe au premier plan, titre optionnel) ; lien de vue éphémère vers un Obsidian distant, pour les déploiements en conteneur.

Astuces · Tapez `/obsidian-router:` dans Claude Code → complétion automatique · Chaque commande accepte un argument `vault` (sinon, le vault du workspace) · `vault: "*"` sur `search` / `search_smart` = tous les vaults · Les phrases déclencheuses sont indicatives, Claude reconnaît les variantes · Les résultats d'écriture portent un `clickToOpenUrl` tout fait — un clic ouvre la note dans Obsidian · **Noms des outils selon l'installation** : mêmes outils, trois préfixes — plugin : `mcp__plugin_obsidian-router_router__<tool>` · enregistrement manuel : `mcp__obsidian-router__<tool>` · MCPHub : `mcp__<id>__obsidian-router-<vault>-<tool>` ; les deux premiers coexistent (outils en double) si l'entrée manuelle est conservée · Doc complète : `README.md`, `docs/features/` (13 fiches), `docs/auto-enrichment.md`, `docs/remote-vaults.md` dans le dépôt.